

relevant-when: Open this drawer when writing hierarchical queries, Oracle date operations, or analytic window functions.

Oracle SQL Idioms Reference

1. Analytic Window Functions

Compute aggregates over partition sets without collapsing rows.

```
SELECT
    employee_id, department_id, salary,
    -- Unique row number per partition
    ROW_NUMBER() OVER (PARTITION BY department_id ORDER BY salary DESC) rn,
    -- Rank per partition (gaps in ranks for tied values)
    RANK() OVER (PARTITION BY department_id ORDER BY salary DESC) rk,
    -- Value from the next row
    LEAD(salary, 1) OVER (PARTITION BY department_id ORDER BY salary) next_sal,
    -- Value from the previous row
    LAG(salary, 1) OVER (PARTITION BY department_id ORDER BY salary) prev_sal
FROM employees;

-- String concatenation aggregation
SELECT department_id,
    LISTAGG(last_name, ';' ) WITHIN GROUP (ORDER BY last_name) emp_list
FROM employees
GROUP BY department_id;
```

2. Oracle Date Math

Date and timestamp calculations using native functions and interval literals.

```
-- Current system datetime
SELECT SYSDATE FROM dual;

-- Add or subtract months
SELECT ADD_MONTHS(SYSDATE, 3) FROM dual; -- 3 months in future
SELECT ADD_MONTHS(SYSDATE, -1) FROM dual; -- 1 month in past

-- Add or subtract custom intervals
SELECT SYSDATE + INTERVAL '1' DAY FROM dual;
SELECT SYSDATE + INTERVAL '12' HOUR FROM dual;
SELECT SYSDATE + INTERVAL '30' MINUTE FROM dual;
SELECT SYSDATE - INTERVAL '2' YEAR FROM dual;

-- Date difference in days
SELECT (date_end - date_start) AS days_diff FROM events;
```

3. Hierarchical Queries

CONNECT BY Syntax

Traditional Oracle-specific hierarchical syntax.

```
SELECT employee_id, last_name, manager_id, LEVEL,
    SYS_CONNECT_BY_PATH(last_name, ' -> ') path
FROM employees
```

```
START WITH manager_id IS NULL
CONNECT BY PRIOR employee_id = manager_id;
```

Recursive CTE Syntax (Standard ANSI SQL)

Alternative method supported in modern Oracle versions.

```
WITH emp_hierarchy (employee_id, last_name, manager_id, lvl) AS (
  -- Anchor member
  SELECT employee_id, last_name, manager_id, 1
  FROM employees
  WHERE manager_id IS NULL
  UNION ALL
  -- Recursive member
  SELECT e.employee_id, e.last_name, e.manager_id, h.lvl + 1
  FROM employees e
  JOIN emp_hierarchy h ON e.manager_id = h.employee_id
)
SELECT employee_id, last_name, manager_id, lvl FROM emp_hierarchy;
```